

MUP Lab

Microprocessors & Interfacing

Memory System Design in Verilog

A Comprehensive Preparation Guide — From Scratch

What This Guide Covers

- D Flip-Flop design and clocking behaviour
- Building registers from D flip-flops using **generate** blocks
- Decoders (4-to-16 and 5-to-32) for address decoding
- Multiplexers (2-to-1, 32-to-1/2) for output selection
- RAM and ROM hierarchy (32-byte block → 1 KB)
- Memory system with address decoding and Chip Select generation
- Control signals: `RD_n`, `WR_n`, `M_IO_n`
- Key Verilog concepts: **generate**, **initial**, **case**, concatenation

Based on Lab 9 (RAM — 2023-24) and the upcoming Lab circuit diagrams

Contents

1	Overview and Architecture	2
1.1	What Are You Building?	2
1.2	Module Hierarchy	2
2	Essential Verilog Concepts	3
2.1	The <code>generate</code> Block	3
2.1.1	Syntax	3
2.1.2	How it was used in Lab 9	3
2.1.3	For the upcoming lab	4
2.2	Bit Concatenation	4
2.3	ROM Implementation in Verilog	5
2.3.1	Method 1: <code>case</code> statement (combinational)	5
2.3.2	Method 2: Array with <code>initial</code> block (register-file style)	5
2.4	Active-Low Signals and Chip Select Logic	6
3	Module-by-Module Implementation	7
3.1	Module 1: <code>D_ff</code> — D Flip-Flop	7
3.2	Module 2: <code>mux2to1_16bits</code> — 2-to-1 Multiplexer	8
3.3	Module 3: <code>decoder4to16</code> / <code>decoder5to32</code>	8
3.4	Module 4: <code>register16bit</code>	10
3.5	Module 5: <code>mux32to1</code> — 32-to-1 Multiplexer (16-bit wide)	10
3.6	Module 6: <code>ram32byte</code> — Core RAM Block	12
3.7	Module 7: <code>ram1kb</code> — 1 KB RAM (32 blocks of <code>ram32byte</code>)	13
3.8	Module 8: <code>rom32byte</code> and <code>rom1kb</code>	15
3.9	Module 9: <code>memory_system</code> — Top-Level Integration	17
4	Key Concepts Deep Dive	18
4.1	How the Write Path Works (End-to-End)	18
4.2	How the Read Path Works (End-to-End)	18
4.3	Understanding Chip Select Cascading	19
5	Common Mistakes and How to Avoid Them	19
6	Quick Reference: Verilog Syntax Cheat Sheet	19
6.1	Module Declaration	20
6.2	Always Blocks	20
6.3	Instantiation — Named vs. Positional	20
6.4	Common Number Literals	20
6.5	Wire vs. Reg	21
6.6	The <code>generate</code> Block — Full Template	21
7	Lab 9 (2023-24) Solution Summary	21
7.1	What Lab 9 Built	21
7.2	What the Upcoming Lab Extends	22
8	Preparation Checklist	22
9	Glossary	22

1 Overview and Architecture

1.1 What Are You Building?

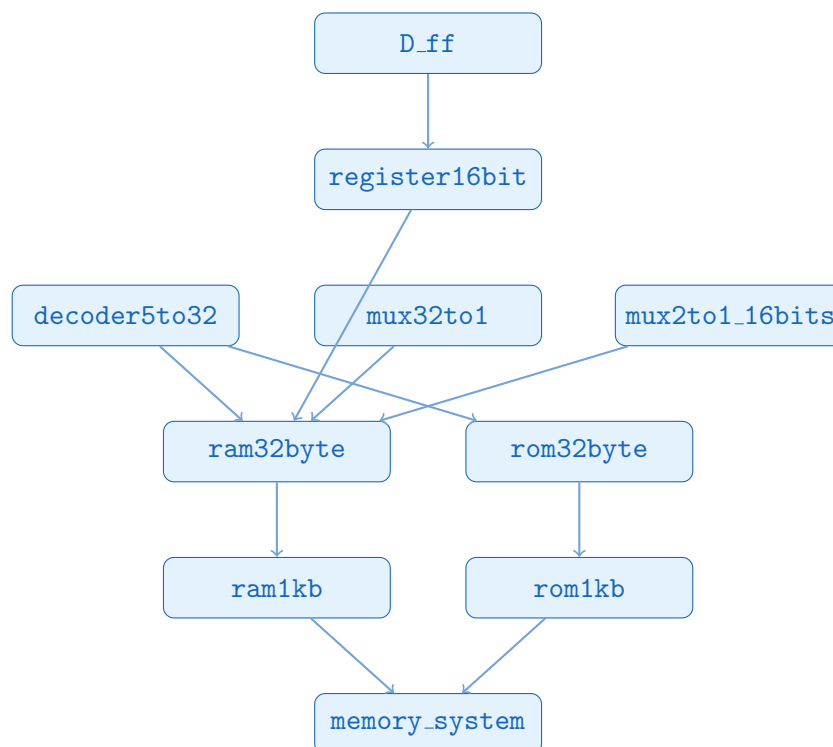
The lab asks you to implement a **memory system** consisting of RAM and ROM chips connected through an address decoder. The system is hierarchical — you build small blocks first and compose them into larger ones.

The final `memory_system` module has the following external interface:

Port	Direction	Description
<code>addr[19:0]</code>	Input	20-bit memory address
<code>RD_n</code>	Input	Read enable (active low)
<code>WR_n</code>	Input	Write enable (active low)
<code>M_IO_n</code>	Input	Memory vs. I/O select
<code>data_out[15:0]</code>	Output	16-bit read data
<code>clk</code>	Input	System clock
<code>reset</code>	Input	Synchronous reset

1.2 Module Hierarchy

Build modules in this order — each level depends on the one below it.



Differences from the Lab 9 (2023-24) solution: The previous lab built a simple 32-byte RAM with 8-bit registers. The upcoming lab uses **16-bit registers**, a **5-to-32 decoder** (instead of 4-to-16), a **32-to-1 mux** for 16-bit outputs, and extends upward to **RAM 1 KB**, **ROM 1 KB**, and a full **memory_system**. The concepts are

identical — only the widths and hierarchy change.

2 Essential Verilog Concepts

This section covers the Verilog features you need that may be new or need revision from your Digital Design course.

2.1 The generate Block

A **generate** block lets you *instantiate hardware repetitively* using a loop — this is elaborated at compile time, not run time. It is the key construct used to build a register out of many D flip-flops, and a RAM block out of many registers.

2.1.1 Syntax

```

1  genvar i;                                // loop variable (
    elaboration-time only)
2  generate
3      for (i = 0; i < N; i = i + 1) begin : LABEL
4          // Module instantiation or logic here
5          MyModule inst_name (
6              .port1 (signal1[i]),
7              .port2 (signal2[i])
8          );
9      end
10 endgenerate

```

Listing 1: generate block syntax

Common mistakes with generate:

- **genvar** must be declared *outside* the **generate** block (or inside in SystemVerilog — use Verilog-2001 style for this lab).
- Always give the loop a label (**begin : LABEL**) — some simulators require it for named instance access.
- You *cannot* use a **genvar** as a runtime signal — it only exists during elaboration.

2.1.2 How it was used in Lab 9

The **register8bit** module used **generate** to instantiate 8 D flip-flops:

```

1  'include "D_ff.v"
2  module register8bit(
3      input clk, input chipSelect, input reset,
4      input regWrite, input enable,
5      input  [7:0] inR,
6      output [7:0] outR
7  );
8      genvar i;

```

```

9     generate
10         for (i = 0; i < 8; i = i + 1)
11             D_ff dff (clk, chipSelect, reset, regWrite,
12                     enable, inR[i], outR[i]);
13     endgenerate
14 endmodule

```

Listing 2: register8bit from Lab 9 solution — generate usage

2.1.3 For the upcoming lab

You will do the same for `register16bit` (16 flip-flops), and use `generate` when instantiating 32 registers inside `ram32byte`:

```

1  'include "D_ff.v"
2  module register16bit(
3      input clk, input chipSelect, input reset,
4      input regWrite, input enable,
5      input  [15:0] inR,
6      output [15:0] outR
7  );
8      genvar i;
9      generate
10         for (i = 0; i < 16; i = i + 1)
11             D_ff dff (clk, chipSelect, reset, regWrite,
12                     enable, inR[i], outR[i]);
13     endgenerate
14 endmodule

```

Listing 3: register16bit — template

2.2 Bit Concatenation

The `{}` operator joins bit vectors. It is used extensively when reading two adjacent bytes as one 16-bit word:

```

1  wire [7:0]  high_byte, low_byte;
2  wire [15:0] word;
3
4  assign word = {high_byte, low_byte}; // high_byte is bits
   [15:8]
5
   // low_byte is bits
   [7:0]
6
7  // In a case (mux) statement:
8  5'd0: muxOut = {in0, in1}; // Big-Endian: lower address =
   higher byte

```

Listing 4: Concatenation examples

Big-Endian memory layout (used in Lab 9 and the upcoming lab):

If you write 16'hABCD to address 0:

Byte address 0 → 0xAB (high byte, data[15:8])

Byte address 1 → 0xCD (low byte, data[7:0])

When reading, muxOut = {mem[addr], mem[addr+1]} reconstructs the original word.

2.3 ROM Implementation in Verilog

A ROM has no write port — its contents are fixed. There are two common ways to implement it.

2.3.1 Method 1: case statement (combinational)

```

1 module rom32byte(
2     input  chipSelect, outputEnable,
3     input  [4:0] address,
4     output reg [15:0] dataOut
5 );
6     always @(*) begin
7         if (chipSelect && outputEnable) begin
8             case (address)
9                 5'd0: dataOut = 16'hDEAD;
10                5'd1: dataOut = 16'hBEEF;
11                5'd2: dataOut = 16'h1234;
12                // ... all 32 entries
13                default: dataOut = 16'h0000;
14            endcase
15        end else begin
16            dataOut = 16'h0000;
17        end
18    end
19 endmodule

```

Listing 5: ROM using a case statement

2.3.2 Method 2: Array with initial block (register-file style)

```

1 module rom32byte(
2     input  chipSelect, outputEnable,
3     input  [4:0] address,
4     output [15:0] dataOut
5 );
6     reg [15:0] mem [0:31];    // 32 locations, 16 bits each
7
8     initial begin
9         mem[0] = 16'hDEAD;
10        mem[1] = 16'hBEEF;
11        mem[2] = 16'h1234;

```

```

12      // ... fill all 32 entries
13      end
14
15      // Read port (combinational, guarded by control signals)
16      assign dataOut = (chipSelect && outputEnable) ? mem[address
17      ] : 16'h0000;
18  endmodule

```

Listing 6: ROM using an initial block to pre-load memory

Which method to use? Both work for simulation. The `initial` block approach is cleaner for large ROMs. The `case` approach mirrors what was done in Lab 9 for decoders, so it may feel more familiar. The circuit diagram suggests building `rom32byte` as “ 32×16 -bit ROM” — the `initial` block maps naturally to this.

2.4 Active-Low Signals and Chip Select Logic

In the upcoming memory system, control signals use an **active-low convention** (suffix `_n`):

Signal	Active when	Meaning
<code>RD_n</code>	Low (0)	Read from memory
<code>WR_n</code>	Low (0)	Write to memory
<code>M_IO_n</code>	Low (0)	Address is in memory space (not I/O)

To drive a module’s active-high `chipSelect`, you invert the address decode:

```

1  // Example: RAM occupies addresses 0x00000 - 0x003FF (lower 1
   // KB)
2  //           ROM occupies addresses 0x00400 - 0x007FF (next 1 KB
   // )
3  // addr[19:0] is the full address bus
4
5  wire ram_cs = (addr[19:10] == 10'b00_0000_0000) && !M_IO_n;
6  wire rom_cs = (addr[19:10] == 10'b00_0000_0001) && !M_IO_n;
7
8  // Active-low read/write for the submodules
9  wire read_en = !RD_n;
10 wire write_en = !WR_n;

```

Listing 7: Chip Select generation from address decoding

How address decoding works (big picture):

With a 20-bit address bus (`A[19:0]`), you can address up to 1 MB. The upper address bits select *which chip*, and the lower bits select *which location within that chip*. For a 1 KB RAM: the lower 10 bits (`addr[9:0]`) address the location, and `addr[19:10]` identifies the chip.

3 Module-by-Module Implementation

3.1 Module 1: D_ff — D Flip-Flop

Port	Dir	Description
clk	In	Clock (positive edge triggered)
chipSelect	In	Enables this chip
reset	In	Synchronous reset (active high)
regWrite	In	Write enable
enable	In	From decoder — selects this specific FF
d	In	Data input
q	Out	Data output

Behaviour:

1. Everything happens only at the **positive clock edge**.
2. If chipSelect && reset: output $q \leftarrow 0$.
3. Else if chipSelect && enable && regWrite: output $q \leftarrow d$.
4. Otherwise: q is unchanged (holds its value).

```

1 module D_ff(
2     input clk,
3     input chipSelect,
4     input reset,
5     input regWrite,
6     input enable,
7     input d,
8     output reg q
9 );
10 always @(posedge clk) begin
11     if (chipSelect == 1'b1 && reset == 1'b1)
12         q <= 0;
13     else if (regWrite == 1'b1 && enable == 1'b1 &&
14             chipSelect == 1'b1)
15         q <= d;
16     // implicit else: q holds value
17 end
18 endmodule

```

Listing 8: D_ff — complete implementation (from Lab 9 solution)

Priority: Reset takes priority over write. Notice that the `if` for reset comes first — if both `reset` and `regWrite` are high simultaneously, reset wins.

3.2 Module 2: mux2to1_16bits — 2-to-1 Multiplexer

Port	Dir	Description
in0[15:0]	In	Input when select = 0
in1[15:0]	In	Input when select = 1
select	In	Selection bit
muxOut[15:0]	Out	Selected input

Usage in the memory: This mux gates the output. When outputEnable = 0 or chipSelect = 0, in0 = 16'b0 is selected so memOut goes to zero (the bus is released).

```

1 module mux2to1_16bits(
2     input  [15:0] in0,
3     input  [15:0] in1,
4     input          select,
5     output reg [15:0] muxOut
6 );
7     always @(in0, in1, select) begin
8         case (select)
9             1'b0: muxOut = in0;
10            1'b1: muxOut = in1;
11        endcase
12    end
13 endmodule

```

Listing 9: mux2to1_16bits implementation

```

1 // outputEnable & chipSelect drives the select:
2 // if either is 0, muxOut = in0 = 16'b0 (bus floats to 0)
3 // if both are 1, muxOut = in1 = actual memory data
4 mux2to1_16bits muxRead (
5     .in0      (16'b0),
6     .in1      (memReadOut),
7     .select   (outputEnable & chipSelect),
8     .muxOut   (memOut)
9 );

```

Listing 10: How it is instantiated in the RAM module

3.3 Module 3: decoder4to16 / decoder5to32

A decoder takes an n -bit input and asserts exactly one of 2^n output lines. It is used to select which memory register the address points to.

Module	Input	Output
decoder4to16	destReg[3:0]	decOut[15:0]
decoder5to32	selectIn[4:0]	decOut[31:0]

```

1 module decoder4to16(
2     input  [3:0] destReg,

```

```

3     output reg [15:0] decOut
4 );
5     always @(destReg) begin
6         case (destReg)
7             4'd0: decOut = 16'b0000_0000_0000_0001;
8             4'd1: decOut = 16'b0000_0000_0000_0010;
9             4'd2: decOut = 16'b0000_0000_0000_0100;
10            4'd3: decOut = 16'b0000_0000_0000_1000;
11            4'd4: decOut = 16'b0000_0000_0001_0000;
12            4'd5: decOut = 16'b0000_0000_0010_0000;
13            4'd6: decOut = 16'b0000_0000_0100_0000;
14            4'd7: decOut = 16'b0000_0000_1000_0000;
15            4'd8: decOut = 16'b0000_0001_0000_0000;
16            4'd9: decOut = 16'b0000_0010_0000_0000;
17            4'd10: decOut = 16'b0000_0100_0000_0000;
18            4'd11: decOut = 16'b0000_1000_0000_0000;
19            4'd12: decOut = 16'b0001_0000_0000_0000;
20            4'd13: decOut = 16'b0010_0000_0000_0000;
21            4'd14: decOut = 16'b0100_0000_0000_0000;
22            4'd15: decOut = 16'b1000_0000_0000_0000;
23        endcase
24    end
25 endmodule

```

Listing 11: decoder4to16 — from Lab 9 solution

```

1 module decoder5to32(
2     input  [4:0] selectIn,
3     output reg [31:0] decOut
4 );
5     always @(selectIn) begin
6         case (selectIn)
7             5'd0: decOut = 32'
8                 b0000_0000_0000_0000_0000_0000_0001;
9             5'd1: decOut = 32'
10                b0000_0000_0000_0000_0000_0000_0010;
11            5'd2: decOut = 32'
12                b0000_0000_0000_0000_0000_0000_0100;
13            // ... pattern: decOut = 32'd1 << selectIn
14            5'd31: decOut = 32'
15                b1000_0000_0000_0000_0000_0000_0000;
16            default: decOut = 32'd0;
17        endcase
18    end
19 endmodule

```

Listing 12: decoder5to32 — extend the same pattern to 5 bits / 32 outputs

Shortcut using shift: You can write the decoder body as a single line:

```
1 assign decOut = (32'd1 << selectIn);
```

This is functionally equivalent and much shorter. Use whichever your lab permits.

3.4 Module 4: register16bit

Sixteen D flip-flops wired in parallel. Each bit of `inR[15:0]` connects to one `D_ff`, with a shared `enable` line coming from the decoder.

```
1 'include "D_ff.v"
2 module register16bit(
3     input  clk, chipSelect, reset, regWrite, enable,
4     input  [15:0] inR,
5     output [15:0] outR
6 );
7     genvar i;
8     generate
9         for (i = 0; i < 16; i = i + 1) begin : REG_BITS
10             D_ff dff (
11                 .clk          (clk),
12                 .chipSelect   (chipSelect),
13                 .reset        (reset),
14                 .regWrite     (regWrite),
15                 .enable       (enable),
16                 .d            (inR[i]),
17                 .q            (outR[i])
18             );
19         end
20     endgenerate
21 endmodule
```

Listing 13: register16bit — uses generate to instantiate 16 `D_ff`s

3.5 Module 5: mux32to1 — 32-to-1 Multiplexer (16-bit wide)

Selects one 16-bit word from 32 possible register outputs, based on a 5-bit address.

Port	Dir	Description
in0 ... in31 [15:0]	In	Outputs of the 32 registers
select[4:0]	In	5-bit address
muxOut[15:0]	Out	Selected register output

```
1 module mux32to1(
2     input  [15:0] in0,  in1,  in2,  in3,
3     input  [15:0] in4,  in5,  in6,  in7,
4     input  [15:0] in8,  in9,  in10, in11,
5     input  [15:0] in12, in13, in14, in15,
6     input  [15:0] in16, in17, in18, in19,
7     input  [15:0] in20, in21, in22, in23,
8     input  [15:0] in24, in25, in26, in27,
```

```

9      input  [15:0] in28, in29, in30, in31,
10     input  [4:0]  select,
11     output reg [15:0] muxOut
12 );
13     always @(*) begin
14         case (select)
15             5'd0:  muxOut = in0;
16             5'd1:  muxOut = in1;
17             5'd2:  muxOut = in2;
18             5'd3:  muxOut = in3;
19             5'd4:  muxOut = in4;
20             5'd5:  muxOut = in5;
21             5'd6:  muxOut = in6;
22             5'd7:  muxOut = in7;
23             5'd8:  muxOut = in8;
24             5'd9:  muxOut = in9;
25             5'd10: muxOut = in10;
26             5'd11: muxOut = in11;
27             5'd12: muxOut = in12;
28             5'd13: muxOut = in13;
29             5'd14: muxOut = in14;
30             5'd15: muxOut = in15;
31             5'd16: muxOut = in16;
32             5'd17: muxOut = in17;
33             5'd18: muxOut = in18;
34             5'd19: muxOut = in19;
35             5'd20: muxOut = in20;
36             5'd21: muxOut = in21;
37             5'd22: muxOut = in22;
38             5'd23: muxOut = in23;
39             5'd24: muxOut = in24;
40             5'd25: muxOut = in25;
41             5'd26: muxOut = in26;
42             5'd27: muxOut = in27;
43             5'd28: muxOut = in28;
44             5'd29: muxOut = in29;
45             5'd30: muxOut = in30;
46             5'd31: muxOut = in31;
47             default: muxOut = 16'b0;
48         endcase
49     end
50 endmodule

```

Listing 14: mux32to1 implementation

Difference from Lab 9:

Lab 9 used mux32to2 — selecting **two bytes** (one 16-bit word from two adjacent 8-bit registers) using a 4-bit address. In the upcoming lab, since each register is already 16 bits wide, you use mux32to1 — selecting **one 16-bit register** from 32 using a 5-bit address. The logic is the same; only widths differ.

3.6 Module 6: ram32byte — Core RAM Block

This is the heart of the design. It wires together:

1. A `decoder5to32` to select which register to write.
2. 32 instances of `register16bit` (the storage).
3. A `mux32to1` to select which register to read.
4. A `mux2to1_16bits` to gate the output.

Port	Dir	Description
<code>clk</code>	In	Clock
<code>chipSelect</code>	In	This chip is selected
<code>reset</code>	In	Synchronous reset
<code>writeEnable</code>	In	Write to memory
<code>outputEnable</code>	In	Read from memory
<code>address[4:0]</code>	In	5-bit address (selects 1 of 32 registers)
<code>writeData[15:0]</code>	In	Data to write
<code>memOut[15:0]</code>	Out	Data read out

```

1  'include "decoder5to32.v"
2  'include "register16bit.v"
3  'include "mux32to1.v"
4  'include "mux2to1_16bits.v"
5
6  module ram32byte(
7      input  clk, chipSelect, reset, writeEnable, outputEnable,
8      input  [4:0] address,
9      input  [15:0] writeData,
10     output [15:0] memOut
11 );
12     // Wires for 32 register outputs
13     wire [15:0] Dmem [0:31];
14
15     // Decoder: address -> one-hot enable
16     wire [31:0] decOut;
17     decoder5to32 writeDec (.selectIn(address), .decOut(decOut))
18         ;
19
20     // 32 x 16-bit registers
21     genvar i;
22     generate
23         for (i = 0; i < 32; i = i + 1) begin : RAM_REGS
24             register16bit reg_i (
25                 .clk          (clk),
26                 .chipSelect   (chipSelect),
27                 .reset        (reset),
28                 .regWrite     (writeEnable),
29                 .enable       (decOut[i]),
30                 .inR          (writeData),
31                 .outR         (Dmem[i])

```

```

32         end
33     endgenerate
34
35     // Read mux: select one of 32 registers
36     wire [15:0] memReadOut;
37     mux32to1 muxRead (
38         .in0(Dmem[0]),    .in1(Dmem[1]),    .in2(Dmem[2]),    .in3
39         (Dmem[3]),
40         .in4(Dmem[4]),    .in5(Dmem[5]),    .in6(Dmem[6]),    .in7
41         (Dmem[7]),
42         .in8(Dmem[8]),    .in9(Dmem[9]),    .in10(Dmem[10]),   .
43         in11(Dmem[11]),
44         .in12(Dmem[12]), .in13(Dmem[13]), .in14(Dmem[14]), .
45         in15(Dmem[15]),
46         .in16(Dmem[16]), .in17(Dmem[17]), .in18(Dmem[18]), .
47         in19(Dmem[19]),
48         .in20(Dmem[20]), .in21(Dmem[21]), .in22(Dmem[22]), .
49         in23(Dmem[23]),
50         .in24(Dmem[24]), .in25(Dmem[25]), .in26(Dmem[26]), .
51         in27(Dmem[27]),
52         .in28(Dmem[28]), .in29(Dmem[29]), .in30(Dmem[30]), .
53         in31(Dmem[31]),
54         .select(address),
55         .muxOut(memReadOut)
56     );
57
58     // Output gate: only drive memOut when OE & CS are both
59     // active
60     mux2to1_16bits muxOut_gate (
61         .in0    (16'b0),
62         .in1    (memReadOut),
63         .select (outputEnable & chipSelect),
64         .muxOut (memOut)
65     );
66 endmodule

```

Listing 15: ram32byte — full structural implementation

3.7 Module 7: ram1kb — 1 KB RAM (32 blocks of ram32byte)

32 instances of `ram32byte`, each containing 32 registers of 16 bits = 32×2 bytes = 64 bytes per block. 32 blocks $\times$ 64 bytes = 2048 bytes $\approx$ 2 KB? Actually in the circuit diagram, `ram1kb` uses `address[9:5]` to select the block and `address[4:0]` as the within-block address.

Port	Dir	Description
clk, chipSelect, reset	In	Standard control
outputEnable, writeEnable	In	Read/write enables
address[9:0]	In	10-bit address: [9:5] = block, [4:0] = offset
writeData[15:0]	In	Write data
memOut[15:0]	Out	Read data

```

1  'include "decoder5to32.v"
2  'include "ram32byte.v"
3  'include "mux32to1.v"
4
5  module ram1kb(
6      input  clk, chipSelect, reset, outputEnable, writeEnable,
7      input  [9:0]  address,
8      input  [15:0] writeData,
9      output [15:0] memOut
10 );
11     // Upper 5 bits select which ram32byte block
12     wire [31:0] blockSel;
13     decoder5to32 blockDec (
14         .selectIn (address[9:5]),
15         .decOut    (blockSel)
16     );
17
18     // Output from each block
19     wire [15:0] blockOut [0:31];
20
21     // Instantiate 32 ram32byte blocks
22     genvar i;
23     generate
24         for (i = 0; i < 32; i = i + 1) begin : RAM_BLOCKS
25             ram32byte block_i (
26                 .clk            (clk),
27                 .chipSelect     (blockSel[i] & chipSelect),
28                 .reset          (reset),
29                 .writeEnable    (writeEnable),
30                 .outputEnable   (outputEnable),
31                 .address        (address[4:0]),
32                 .writeData      (writeData),
33                 .memOut         (blockOut[i])
34             );
35         end
36     endgenerate
37
38     // Output mux: select the active block's output
39     mux32to1 outMux (
40         .in0(blockOut[0]), .in1(blockOut[1]), .in2(blockOut
41             [2]), .in3(blockOut[3]),
42         .in4(blockOut[4]), .in5(blockOut[5]), .in6(blockOut
43             [6]), .in7(blockOut[7]),
44         .in8(blockOut[8]), .in9(blockOut[9]), .in10(

```

```

        blockOut[10]), .in11(blockOut[11]),
43      .in12(blockOut[12]), .in13(blockOut[13]), .in14(
        blockOut[14]), .in15(blockOut[15]),
44      .in16(blockOut[16]), .in17(blockOut[17]), .in18(
        blockOut[18]), .in19(blockOut[19]),
45      .in20(blockOut[20]), .in21(blockOut[21]), .in22(
        blockOut[22]), .in23(blockOut[23]),
46      .in24(blockOut[24]), .in25(blockOut[25]), .in26(
        blockOut[26]), .in27(blockOut[27]),
47      .in28(blockOut[28]), .in29(blockOut[29]), .in30(
        blockOut[30]), .in31(blockOut[31]),
48      .select  (address[9:5]),
49      .muxOut  (memOut)
50    );
51  endmodule

```

Listing 16: ram1kb — hierarchical RAM using generate

Key insight — Chip Select cascading:

Notice that each `ram32byte` block receives `chipSelect = blockSel[i] & chipSelect`. This means a block is only active if (a) the upper address bits select *that* block, AND (b) the entire `ram1kb` chip is itself selected. This is the standard way chip selects cascade in hierarchical memory designs.

3.8 Module 8: rom32byte and rom1kb

The ROM modules mirror the RAM hierarchy but have no write port. The circuit diagram shows `rom32byte` as a “ 32×16 -bit ROM” — a fixed lookup table.

```

1  module rom32byte(
2      input  chipSelect, outputEnable,
3      input  [4:0]  address,
4      output [15:0] dataOut
5  );
6      reg [15:0] mem [0:31];
7
8      initial begin
9          // Pre-load ROM contents
10         mem[0]  = 16'h0000;
11         mem[1]  = 16'h1111;
12         mem[2]  = 16'h2222;
13         // ... fill all 32 entries as required by your lab
14         mem[31] = 16'hFFFF;
15     end
16
17     assign dataOut = (chipSelect && outputEnable) ? mem[address
18         ] : 16'h0000;
19 endmodule

```

Listing 17: rom32byte — using initial block


```

1  'include "decoder5to32.v"
2  'include "rom32byte.v"
3  'include "mux32to1.v"
4
5  module rom1kb(
6      input  chipSelect, outputEnable,
7      input  [9:0]  address,
8      output [15:0] dataOut
9  );
10     wire [31:0] blockSel;
11     decoder5to32 blockDec (.selectIn(address[9:5]), .decOut(
        blockSel));
12
13     wire [15:0] blockOut [0:31];
14
15     genvar i;
16     generate
17         for (i = 0; i < 32; i = i + 1) begin : ROM_BLOCKS
18             rom32byte block_i (
19                 .chipSelect  (blockSel[i] & chipSelect),
20                 .outputEnable (outputEnable),
21                 .address      (address[4:0]),
22                 .dataOut      (blockOut[i])
23             );
24         end
25     endgenerate
26
27     mux32to1 outMux (
28         .in0(blockOut[0]), .in1(blockOut[1]), .in2(blockOut
        [2]), .in3(blockOut[3]),
29         .in4(blockOut[4]), .in5(blockOut[5]), .in6(blockOut
        [6]), .in7(blockOut[7]),
30         .in8(blockOut[8]), .in9(blockOut[9]), .in10(
        blockOut[10]), .in11(blockOut[11]),
31         .in12(blockOut[12]), .in13(blockOut[13]), .in14(
        blockOut[14]), .in15(blockOut[15]),
32         .in16(blockOut[16]), .in17(blockOut[17]), .in18(
        blockOut[18]), .in19(blockOut[19]),
33         .in20(blockOut[20]), .in21(blockOut[21]), .in22(
        blockOut[22]), .in23(blockOut[23]),
34         .in24(blockOut[24]), .in25(blockOut[25]), .in26(
        blockOut[26]), .in27(blockOut[27]),
35         .in28(blockOut[28]), .in29(blockOut[29]), .in30(
        blockOut[30]), .in31(blockOut[31]),
36         .select  (address[9:5]),
37         .muxOut   (dataOut)
38     );
39 endmodule

```

Listing 18: rom1kb — same structure as ram1kb but using rom32byte blocks

3.9 Module 9: memory_system — Top-Level Integration

This is the final module that wires ram1kb and rom1kb to the processor bus.

```

1  'include "ram1kb.v"
2  'include "rom1kb.v"
3
4  module memory_system(
5      input  clk, reset,
6      input  [19:0] addr,    // A[19:1] from 8086-style bus (word
                             // addressed)
7      input          RD_n,    // Active-low read
8      input          WR_n,    // Active-low write
9      input          M_IO_n,  // Active-low: memory (not I/O)
                             // access
10     output [15:0] data_out
11 );
12     // Convert active-low signals
13     wire mem_access = ~M_IO_n;
14     wire read_en    = ~RD_n;
15     wire write_en   = ~WR_n;
16
17     // Address decode: upper bits determine which chip
18     // Example mapping (adjust to your lab's specified address
19     // map):
20     // RAM: addr[19:10] == 10'h000 -> 0x00000 - 0x003FF
21     // ROM: addr[19:10] == 10'h001 -> 0x00400 - 0x007FF
22     wire ram_cs = mem_access && (addr[19:10] == 10'h000);
23     wire rom_cs = mem_access && (addr[19:10] == 10'h001);
24
25     wire [15:0] ram_out, rom_out;
26
27     ram1kb ram (
28         .clk          (clk),
29         .chipSelect    (ram_cs),
30         .reset         (reset),
31         .outputEnable  (read_en),
32         .writeEnable   (write_en),
33         .address        (addr[9:0]),
34         .writeData      (16'b0),    // connect to data bus in
35         // real system
36         .memOut         (ram_out)
37     );
38
39     rom1kb rom (
40         .chipSelect    (rom_cs),
41         .outputEnable  (read_en),
42         .address        (addr[9:0]),
43         .dataOut        (rom_out)
44     );
45
46     // Output mux: only one chip drives the bus at a time

```

```

45     assign data_out = ram_cs ? ram_out :
46                           rom_cs ? rom_out :
47                           16'b0;
48 endmodule

```

Listing 19: memory_system — top-level with address decode and CS generation

Check the address map in your lab spec! The exact addresses assigned to RAM and ROM are specified per lab. The example above uses a placeholder. Always read the address decode requirements carefully — the upper bits of the address bus tell you which chip is selected, and you must match them exactly.

4 Key Concepts Deep Dive

4.1 How the Write Path Works (End-to-End)

1. Processor asserts `addr`, `writeData`, drives `WR_n` low, `M_IO_n` low.
2. `memory_system` decodes upper address bits → asserts `ram_cs` or `rom_cs`.
3. `ram1kb`: `decoder5to32` on `addr[9:5]` → asserts one of 32 `ram32byte` chip selects.
4. Active `ram32byte`: `decoder5to32` on `addr[4:0]` → asserts one of 32 `register16bit` enables.
5. Active `register16bit`: all 16 `D_ff` instances capture `writeData[i]` at the next positive clock edge (because `enable = decOut[i] = 1`, `regWrite = writeEnable = 1`, `chipSelect = 1`).
6. All other registers: `enable = 0` → their `D_ffs` hold their value.

4.2 How the Read Path Works (End-to-End)

1. Processor asserts `addr`, drives `RD_n` low.
2. Address decode → chip select to RAM or ROM.
3. In `ram32byte`: all 32 `register16bit` outputs (`Dmem[0..31]`) are always available.
4. `mux32to1` selects the correct register using `addr[4:0]`.
5. `mux2to1_16bits` gates the output: passes `memReadOut` only when `outputEnable & chipSelect = 1`; otherwise outputs `16'b0` (releases the bus).
6. Read is *asynchronous* (combinational) — unlike writes which are edge-triggered.

Why is read asynchronous but write synchronous?

Registers (D flip-flops) *capture* data at the clock edge. But *reading* from the stored `q` output is just a wire — the mux connects combinatorially to the register outputs. So reads can happen any time; writes only take effect at the next rising clock edge.

4.3 Understanding Chip Select Cascading

Level	CS formula
memory_system	ram_cs = ~M_IO_n && addr[19:10] == RAMBASE
ram1kb	receives ram_cs as its chipSelect
ram32byte <i>i</i>	chipSelect = ram1kb_cs && blockSel[i]
register16bit <i>j</i>	enable = decOut[j] (within its block's CS)
D_ff	chipSelect && enable && regWrite → write

5 Common Mistakes and How to Avoid Them

1. Forgetting chipSelect in D_ff

The reset condition is `chipSelect && reset`, NOT just `reset`. If you reset without checking `chipSelect`, you will reset every chip in the system whenever `reset` goes high, even chips not selected.

2. Using = instead of <= in sequential logic

Inside `always @(posedge clk)` blocks, always use non-blocking assignment `<=`. Using `=` (blocking) inside sequential blocks causes race conditions.

3. Missing default in case statements

Without a `default`, the synthesizer may infer latches for combinational `always` blocks. Always include `default: out = 0;` (or appropriate value).

4. Big-Endian byte order

When packing two bytes into a 16-bit word: `{highByte, lowByte}` puts `highByte` in `[15:8]`. Mixing this up will give you the correct value stored in the wrong byte order, and reads will return garbage.

5. Connecting decoded enable to enable vs. chipSelect

The decoder output goes to the `enable` port of each register, NOT `chipSelect`. The `chipSelect` of each register is the *block's* chip select — it comes from the next level up (e.g., `ram32byte's` chip select). Mixing these up will cause all registers to respond to every write.

6. Active-low vs. active-high confusion

The memory system uses active-low signals (`RD_n`, `WR_n`, `M_IO_n`) from the bus, but the internal modules use active-high (`outputEnable`, `writeEnable`, `chipSelect`). Always invert at the top level: `wire oe = ~RD_n;`

6 Quick Reference: Verilog Syntax Cheat Sheet

6.1 Module Declaration

```

1 module my_module(
2     input  wire clk,           // single-bit input
3     input  wire [7:0] data,    // 8-bit input bus
4     output reg [15:0] out      // 16-bit registered output
5 );
6     // body
7 endmodule

```

6.2 Always Blocks

```

1 // Sequential logic (clocked)
2 always @(posedge clk) begin
3     q <= d;                    // non-blocking assignment
4 end
5
6 // Combinational logic
7 always @(*) begin              // @(*) = sensitivity to all inputs
8     case (sel)
9         2'b00: out = a;
10        2'b01: out = b;
11        default: out = 0;
12    endcase
13 end

```

6.3 Instantiation — Named vs. Positional

```

1 // Positional (order matters, error-prone for many ports)
2 D_ff dff0 (clk, cs, rst, rw, en, d, q);
3
4 // Named (recommended for clarity)
5 D_ff dff0 (
6     .clk          (clk),
7     .chipSelect   (cs),
8     .reset         (rst),
9     .regWrite      (rw),
10    .enable        (en),
11    .d              (data_in),
12    .q              (data_out)
13 );

```

6.4 Common Number Literals

```

1 4'b1010          // 4-bit binary: 1010
2 8'hFF            // 8-bit hex: 255
3 16'd1024         // 16-bit decimal: 1024
4 32'b0            // 32-bit all zeros

```

```
5 1'b0, 1'b1 // single-bit 0 and 1
```

6.5 Wire vs. Reg

```
1 wire w; // driven by assign or module output
2 reg r; // driven by always block; does NOT mean a
    physical register!
3 // inside always @(*), 'reg' is combinational
    logic
4
5 assign w = a & b; // continuous assignment (combinational)
```

6.6 The generate Block — Full Template

```
1 genvar i;
2 generate
3     for (i = 0; i < N; i = i + 1) begin : UNIQUE_LABEL
4         // Instantiate submodule or write logic here
5         // Use i to index signals: signal[i], array_name[i]
6         SubModule inst (
7             .port_a (bus_a[i]),
8             .port_b (bus_b[i*(WIDTH)-1 : i*(WIDTH) - WIDTH])
9         );
10    end
11 endgenerate
```

7 Lab 9 (2023-24) Solution Summary

This section summarises the actual solution from Lab 9 for reference. Your lab extends this — understanding it fully is half the battle.

7.1 What Lab 9 Built

A 32-byte RAM where:

- Each **register is 8 bits** (one byte), holding 32 bytes total.
- **16 memory locations** accessible at a time (each location = 2 bytes = 16 bits).
- A **4-bit address** selects one of 16 locations.
- A **decoder4to16** activates the correct pair of 8-bit registers.
- A **mux32to2** selects the correct pair of bytes and concatenates them.

7.2 What the Upcoming Lab Extends

Aspect	Lab 9 (2023-24)	Upcoming Lab
Register width	8-bit	16-bit
Registers per block	32	32
Address bits (block)	4-bit	5-bit
Decoder	4-to-16	5-to-32
Read mux	32-to-2 (bytes)	32-to-1 (words)
Hierarchy depth	1 level (RAM)	4 levels (reg → block → 1KB → system)
ROM	Not present	Added: <code>rom32byte</code> , <code>rom1kb</code>
Top-level	<code>ram</code>	<code>memory_system</code>
Control signals	Active-high	Mixed: active-low bus, active-high internal

8 Preparation Checklist

Use this checklist before going into the lab:

- ☐ I can write a D flip-flop with synchronous reset from memory.
- ☐ I understand why `chipSelect` must gate the reset.
- ☐ I can write a `generate` for loop to instantiate N modules.
- ☐ I understand the difference between `enable` (from decoder) and `chipSelect` (from address decode).
- ☐ I can write a `case`-based decoder for n -to- 2^n .
- ☐ I know two ways to implement a ROM in Verilog (`case` and `initial`).
- ☐ I can wire a `mux32to1` module using a `case` statement.
- ☐ I understand Big-Endian byte storage and the `{highByte, lowByte}` concatenation.
- ☐ I understand that reads are combinational and writes are synchronous (edge-triggered).
- ☐ I can derive a Chip Select from an address range.
- ☐ I can convert active-low bus signals to active-high internal signals.
- ☐ I understand how Chip Selects cascade from top-level down to individual D flip-flops.
- ☐ I know the hierarchy: `D_ff` → `register16bit` → `ram32byte` → `ram1kb` → `memory_system`.

9 Glossary

`clk` System clock. All synchronous operations happen on its positive edge.

chipSelect	Enables a specific memory chip. When low (or when not asserted), the chip ignores all operations.
enable	The per-register enable signal, driven by the decoder's one-hot output.
regWrite	Write enable. When high AND chipSelect AND enable are all high, data is written on the next clock edge.
outputEnable	Enables the output port. When low, the chip drives 0 on the data bus (releases it).
RD_n	Active-low read signal from the processor bus.
WR_n	Active-low write signal from the processor bus.
M_IO_n	Active-low signal indicating a memory (not I/O) access.
genvar	An elaboration-time variable used as a loop index in generate blocks.
Big-Endian	Storage format where the most significant byte is at the lowest address.
One-hot	A binary encoding where exactly one bit is asserted at a time. Decoder outputs are one-hot.
Active-low	A signal that performs its function when it is logic 0. Indicated by the _n suffix.